

CSE 344 – System Programming

Homework #4

Multi-Process & Multi-Threaded
Concurrent Log Analysis Pipeline

Student Name: [Ayşe Feyza SERBEST]

Student ID: [220104004052]

Course: CSE344 – System Programming

Homework Number: #4

Date: May 2, 2026

Contents

1	Introduction	2
1.1	Problem Description	2
1.2	Main Challenges	2
1.3	Approach	2
2	System Architecture	4
2.1	Overview	4
2.2	Data Flow Summary	4
3	Shared Memory Layout	5
3.1	Region A – Dispatcher Input Queue	5
3.2	Region B – Per-Level Analysis Buffers ($\times 4$)	5
3.3	Region C – Results Area	6
3.4	Region D – High-Priority Buffer	6
4	Lock Ordering	7
4.1	Global Acquisition Order	7
4.2	Deadlock-Freedom Argument	7
5	Thread Lifecycle	8
5.1	Reader Thread Lifecycle	8
5.2	Worker Thread Lifecycle (Analyzer)	8
6	TLS Design	9
6.1	API Usage	9
6.2	Why the Flush Must Happen in the Destructor, Not at the Barrier	9
7	Dispatcher Logic	10
7.1	EOF Marker Protocol	10
7.2	Handling Multiple Levels Simultaneously	10
7.3	Priority Routing to Region D	10
8	Termination Chain	11
9	Test Scenarios	12
9.1	Test 1 – Baseline	12
9.2	Test 2 – Multi-keyword	12
9.3	Test 3 – Single Thread	13
9.4	Test 4 – High Concurrency (Determinism)	14
9.5	Test 5 – No Priority Matches	15
9.6	Test 6 – Valgrind	16
9.7	Test 7 – Ctrl+C (SIGINT)	17
10	Challenges & Solutions	18
10.1	Challenge 1: Region D Deadlock	18
10.2	Challenge 2: TLS Destructor Pointer Access	18
10.3	Challenge 3: Watchdog Elapsed Counter Overflow	18

1. Introduction

1.1. Problem Description

This homework requires implementing a **Concurrent Log Analysis Pipeline** in C on a POSIX-compatible system. The system reads multiple log files simultaneously, classifies log entries by severity level (ERROR, WARN, INFO, DEBUG), searches for user-specified keywords with level-weighted scoring, identifies high-priority sources through a Dispatcher stage, and produces a structured summary report.

The system must combine two concurrency paradigms:

- **Multi-process architecture:** `fork()`, shared memory via `mmap()`, and POSIX semaphores for inter-process communication.
- **POSIX multi-threading:** `pthread`s, mutexes, condition variables, barriers, and thread-local storage (TLS) within each process.

1.2. Main Challenges

1. **Synchronizing process-shared primitives:** All mutexes and condition variables in shared memory must be initialized with `PTHREAD_PROCESS_SHARED` before any `fork()` call.
2. **Producer-consumer with multiple producers:** Multiple Reader processes write to Region A simultaneously; the Dispatcher must correctly track EOF markers from each reader for each severity level.
3. **TLS destructor ordering:** TLS data must be flushed to Region C in the destructor (at thread exit), *not* at the barrier, to satisfy the assignment's required execution order.
4. **Region D deadlock prevention:** The Aggregator must drain Region D concurrently with waiting for Analyzer semaphores; otherwise the Dispatcher blocks when Region D fills up.
5. **Clean SIGINT handling:** The signal handler must use only async-signal-safe operations; cleanup (SIGTERM/SIGKILL to children, `munmap`, `sem_destroy`) must complete within a 5-second deadline.

1.3. Approach

The implementation was developed incrementally:

1. Shared memory layer (`shm.c`) with all four regions.
2. Process fork architecture (`main.c`) with pipes for the Watchdog.
3. Reader process with internal bounded buffer, reader threads, and a parser thread.
4. Dispatcher routing and EOF forwarding logic.
5. Analyzer processes with TLS, barrier, and `gettid()` reporting.

6. Aggregator with parallel Region D draining and atomic binary output.
7. Watchdog thread using `select()` on pipe read-ends.

2. System Architecture

2.1. Overview

The system consists of 7 process/thread types communicating through 4 shared memory regions and per-reader pipes.

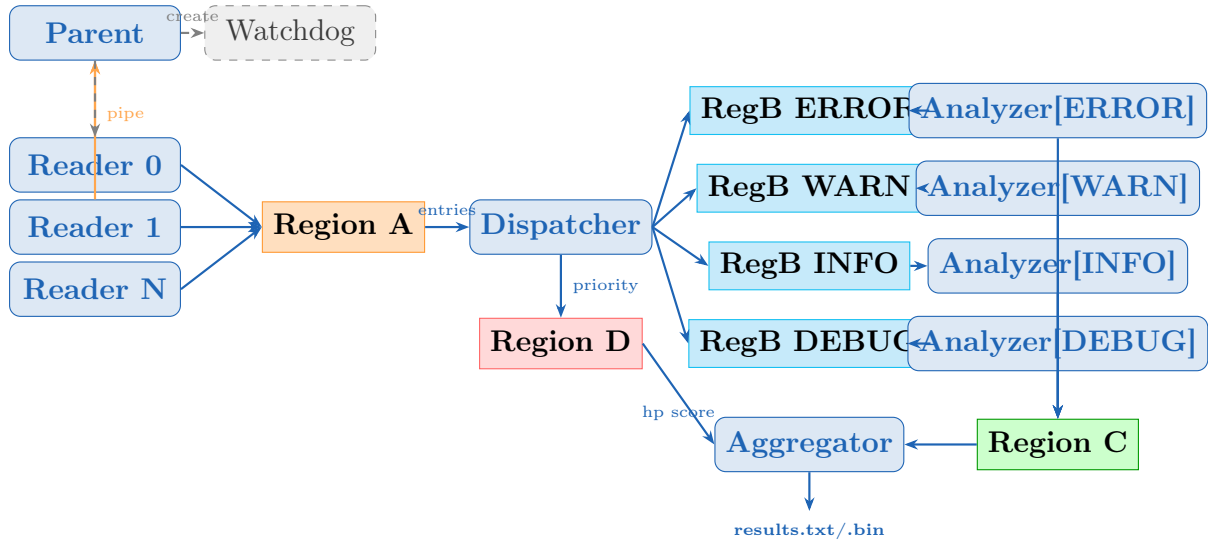


Figure 1: System architecture: 7 process/thread types, 4 shared memory regions, and pipe communication channels

2.2. Data Flow Summary

1. Each **Reader Process** spawns T reader threads that divide the log file into byte ranges, parse lines into `t_log_entry` structs, and push them into an internal bounded buffer. A single parser thread drains this buffer and writes entries to **Region A**. After all entries are sent, the parser thread inserts one EOF marker per severity level into Region A.
2. The **Dispatcher Process** reads from Region A. It routes each entry to the matching per-level buffer in **Region B** and, if the entry's SOURCE field appears in the priority filter, copies it to **Region D** as well.
3. Each **Analyzer Process** (one per level) consumes its Region B buffer using W worker threads. Workers accumulate keyword scores in TLS; on thread exit the TLS destructor flushes results to **Region C**. After a barrier, a `sem_post` signals the Aggregator.
4. The **Aggregator Process** waits for all four semaphores from Region C while simultaneously draining Region D in a helper thread. It then sorts results, writes `results.txt`, and atomically renames a `.tmp` binary file to `results.bin`.
5. The **Watchdog Thread** (inside the parent) uses `select()` on all Reader pipe read-ends with a 3-second timeout, printing progress snapshots to `stderr`.

3. Shared Memory Layout

All four regions are allocated with `mmap(MAP_SHARED|MAP_ANONYMOUS)` in the parent process *before* any `fork()` call. Child processes inherit the mappings and access the same physical pages.

3.1. Region A – Dispatcher Input Queue

```

1 typedef struct s_region_a {
2     pthread_mutex_t  mutex;           //
        PTHREAD_PROCESS_SHARED
3     pthread_cond_t   not_full;       //
        PTHREAD_PROCESS_SHARED
4     pthread_cond_t   not_empty;      //
        PTHREAD_PROCESS_SHARED
5     int               eof_count_per_level[4]; // incremented by each
        Reader's parser thread
6     int               total_readers;   // set by parent before
        fork
7     int               head, tail, count, capacity;
8     t_log_entry       data[];         // flexible array member
9 } t_region_a;

```

Listing 1: Region A struct definition

Field	Protection / Role
<code>mutex</code>	Held by any Reader parser thread writing an entry, or by the Dispatcher reading one. Released before writing to Region B.
<code>not_full</code>	Reader parser thread waits here when <code>count == capacity</code> .
<code>not_empty</code>	Dispatcher waits here (with <code>timedwait</code>) when <code>count == 0</code> .
<code>eof_count_per_level</code>	Incremented atomically under <code>mutex</code> by each Reader's parser thread when it places an EOF marker.

3.2. Region B – Per-Level Analysis Buffers ($\times 4$)

```

1 typedef struct s_region_b {
2     pthread_mutex_t  mutex;           // PTHREAD_PROCESS_SHARED
3     pthread_cond_t   not_full;       // PTHREAD_PROCESS_SHARED
4     pthread_cond_t   not_empty;      // PTHREAD_PROCESS_SHARED
5     int               eof_posted;    // set to 1 by Dispatcher when
        EOF forwarded
6     int               head, tail, count, capacity;
7     t_log_entry       data[];
8 } t_region_b;

```

Listing 2: Region B struct definition

Single-writer constraint: only the Dispatcher writes to Region B. Each Analyzer reads from exactly one buffer (`shm->b[level_index]`). This eliminates write-side contention; only read-side mutex is needed.

3.3. Region C – Results Area

```

1 typedef struct s_level_result {
2     char    level[8];
3     long    total_entries;
4     double   total_weighted_score;
5     double   per_keyword_score[MAX_KEYWORDS];
6     double   per_thread_score[MAX_WORKERS];
7     char     top_source[3][MAX_SOURCE_LEN];
8     long     top_source_hits[3];
9     int      ready;
10 } t_level_result;
11
12 typedef struct s_region_c {
13     pthread_mutex_t  mutex;                // PTHREAD_PROCESS_SHARED
14     pthread_cond_t   result_ready;         // PTHREAD_PROCESS_SHARED
15     sem_t             sem[4];              // process-shared, pshared
16                                     =1
17     t_level_result    results[4];
18     double            high_priority_score; // written by Aggregator,
19                                     read by parent
20 } t_region_c;

```

Listing 3: Region C struct definition

Field	Protection / Role
mutex	Held by TLS destructor when flushing scores; also used by Aggregator for pthread_cond_timedwait.
sem[4]	Each Analyzer posts sem[i] after joining all workers. Aggregator calls sem_timedwait for each.
results[4]	Written exclusively by one Analyzer per slot (no contention between Analyzers).

3.4. Region D – High-Priority Buffer

```

1 typedef struct s_region_d {
2     pthread_mutex_t  mutex;                // PTHREAD_PROCESS_SHARED
3     pthread_cond_t   not_full;             // PTHREAD_PROCESS_SHARED
4     pthread_cond_t   not_empty;           // PTHREAD_PROCESS_SHARED
5     int              dispatcher_done;      // set to 1 by Dispatcher on
6                                     exit
7     int              head, tail, count, capacity;
8     t_log_entry       data[];
9 } t_region_d;

```

Listing 4: Region D struct definition

The `dispatcher_done` flag tells the Aggregator's helper thread to stop blocking once the buffer is empty: if `count == 0 && dispatcher_done == 1`, the loop exits. Without this flag the helper thread would wait indefinitely.

4. Lock Ordering

4.1. Global Acquisition Order

The following total order is enforced throughout the implementation:

Region A $\prec$ Region B $\prec$ Region C $\prec$ Region D

Lock acquired	By whom	What is held before
Region A mutex	Reader parser thread, Dispatcher	Nothing
Region B mutex[i]	Dispatcher	Region A mutex is released first
Region C mutex	TLS destructor, Analyzer (post-join)	Nothing (destructors run after barrier)
Region D mutex	Dispatcher, Aggregator helper thread	Region B mutex is released first

4.2. Deadlock-Freedom Argument

A deadlock requires a cycle in the lock-wait graph. We argue no cycle exists:

1. The Dispatcher acquires Region A mutex, pops one entry, **releases** Region A mutex, then acquires Region B or Region D mutex. It never holds two region mutexes simultaneously.
2. The Reader's parser thread holds Region A mutex only during `region_a_push()`; it never acquires any other mutex while Region A is held.
3. TLS destructors run after `pthread_barrier_wait` returns and the thread calls **return**. At that point no Region B mutex is held. The destructor acquires only Region C mutex.
4. The Aggregator helper thread acquires only Region D mutex.
5. The Aggregator main thread acquires only Region C mutex (for `pthread_cond_timedwait`).

Because no thread ever holds a lower-ranked lock while waiting for a higher-ranked one, and the acquisition order is strictly $A \rightarrow B \rightarrow C/D$ (never in reverse), the lock-wait graph is acyclic and deadlock is impossible.

5. Thread Lifecycle

5.1. Reader Thread Lifecycle

- Step 1. Creation.** `run_reader()` calls `pthread_create()` with a `t_reader_thread_args` struct containing the thread's byte range [`start_byte`, `end_byte`).
- Step 2. Work loop.** The thread opens the log file, seeks to `find_line_start(start_byte)`, and reads lines with `fgets()` until `ftell(fp) >= end_byte`. Each successfully parsed `t_log_entry` is pushed into the internal buffer via `ibuf_push()` (blocks on `not_full` if buffer is full). Every 50 lines a heartbeat is written to the pipe.
- Step 3. Exit.** The thread calls `ibuf_reader_done()`, which decrements `active_readers`. When the last reader decrements to zero, `ibuf.done = 1` is set and the parser thread is signalled via `not_empty`.
- Step 4. Join.** `run_reader()` calls `pthread_join()` for every reader thread, then joins the parser thread.

5.2. Worker Thread Lifecycle (Analyzer)

- Step 1. Creation.** `run_analyzer()` calls `pthread_create()` with a `t_worker_args` struct. A shared `pthread_barrier_t` initialized with count `W` is passed to all workers.
- Step 2. TLS allocation.** The thread allocates `t_tls_data` with `calloc()` and registers it via `pthread_setspecific(key, tls)`. Pointers to `region_c` and `args` are stored inside the TLS struct so the destructor can access them without extra parameters.
- Step 3. Work loop.** The thread pops entries from Region B using `pthread_cond_timedwait` and accumulates keyword scores in TLS: `tls->keyword_scores[i] += count * weight`.
- Step 4. Barrier.** After the loop exits (`eof_posted == 1` and buffer empty), the thread calls `pthread_barrier_wait()`. All `W` workers rendezvous here before proceeding.
- Step 5. Return & TLS destructor.** The thread returns `NULL`. `pthread_exit()` is called implicitly, which triggers the `tls_destructor`. The destructor:
- Acquires Region C mutex.
 - Flushes `keyword_scores`, `total_score`, `entry_count`, and top-3 sources to `region_c->results[lv]`.
 - Releases Region C mutex.
 - Frees the TLS allocation.
- Step 6. Join.** `run_analyzer()` calls `pthread_join()` for all workers. When all joins complete, all destructors have run and Region C is populated. Only then does `sem_post()` signal the Aggregator.

6. TLS Design

6.1. API Usage

```

1 // 1. Key created once per Analyzer process
2 pthread_key_t tls_key;
3 pthread_key_create(&tls_key, tls_destructor); // destructor
   registered here
4
5 // 2. Each worker thread allocates and registers its TLS value
6 t_tls_data *tls = calloc(1, sizeof(t_tls_data));
7 tls->region_c_ptr = warg->region_c; // needed by destructor
8 tls->args_ptr      = warg->args;    // needed by destructor
9 pthread_setspecific(tls_key, tls);
10
11 // 3. Thread accumulates scores in its private TLS (no mutex needed
   )
12 tls->keyword_scores[i] += (double)(hits * weight);
13
14 // 4. On thread exit, destructor is called automatically
15 static void tls_destructor(void *ptr) {
16     t_tls_data *tls = (t_tls_data *)ptr;
17     pthread_mutex_lock(&tls->region_c_ptr->mutex);
18     // flush all scores to Region C
19     pthread_mutex_unlock(&tls->region_c_ptr->mutex);
20     free(tls);
21 }

```

Listing 5: TLS key creation and usage in Analyzer

6.2. Why the Flush Must Happen in the Destructor, Not at the Barrier

The barrier synchronises all W workers at a single point—every thread reaches `pthread_barrier_wait()` before any of them proceeds. *However*, the barrier does not terminate the threads; each thread simply continues executing after the barrier returns. The Thread-Local Storage (TLS) destructor is invoked by the pthreads runtime *when the thread calls `pthread_exit()`*—which happens as the thread function returns after the barrier. The execution sequence is defined as follows:

work loop → barrier_wait → return NULL → destructor runs → join returns

If we flushed TLS data at the barrier, the destructor would still run later on NULL TLS data (because `pthread_setspecific(key, NULL)` would be needed to avoid a double-flush). More critically, the project requirements specify that the flush must occur in the destructor to demonstrate the correct lifecycle management of `pthread_key_t`. The `pthread_join()` function does not return until the target thread has fully terminated, ensuring all associated destructors have finished execution. Consequently, once the joining loop for all W workers completes, all TLS data is guaranteed to be successfully flushed into Region C.

7. Dispatcher Logic

7.1. EOF Marker Protocol

Each Reader process contains one parser thread. When the parser thread has forwarded all log entries to Region A, it inserts *four* EOF markers—one per severity level—and increments `eof_count_per_level[lv]` for each:

```

1 for (int lv = 0; lv < 4; lv++) {
2     eof_entry.level = (t_level)lv;
3     pthread_mutex_lock(&region_a->mutex);
4     region_a_push(region_a, &eof_entry);
5     region_a->eof_count_per_level[lv]++;    // atomic under mutex
6     pthread_cond_signal(&region_a->not_empty);
7     pthread_mutex_unlock(&region_a->mutex);
8 }

```

Listing 6: Parser thread: sending EOF markers

With N Reader processes, the Dispatcher expects `eof_count_per_level[lv] == N` before forwarding a single EOF marker to Region B for that level:

```

1 if (entry.is_eof) {
2     int lv = (int)entry.level;
3     if (!eof_forwarded[lv]
4         && shm->a->eof_count_per_level[lv] >= total_readers) {
5         send_eof_to_region_b(shm->b[lv]);    // one EOF per level to
6         Analyzer
7         eof_forwarded[lv] = 1;
8         finished_levels++;
9     }
10 }

```

Listing 7: Dispatcher: forwarding EOF to Region B

7.2. Handling Multiple Levels Simultaneously

Because Readers interleave entries across all four levels, the Dispatcher processes EOF markers for different levels in any order. The array `eof_forwarded[4]` tracks which levels have already been forwarded. The outer loop condition `finished_levels < 4` ensures the Dispatcher keeps running until all four levels are complete.

Timeout handling: If Region A is empty and a `pthread_cond_timedwait` expires, the Dispatcher checks whether all four levels have been forwarded. If yes, it jumps to `dispatcher_done`. If not, it refreshes the timeout and retries. This prevents the Dispatcher from exiting prematurely when readers are slow.

7.3. Priority Routing to Region D

For every non-EOF entry, the Dispatcher checks whether the `SOURCE` field appears in `args->priority_sources[]`. If yes, a copy of the entry is pushed to Region D *in addition to* Region B. This is a copy, not a move; the entry is analyzed by both the Analyzer (via Region B) and scored separately by the Aggregator (via Region D).

8. Termination Chain

Below is a step-by-step trace of **normal** termination starting from the last Reader Process exiting.

1. **Last Reader Process exits.** All reader threads have finished. The parser thread has sent all entries and four EOF markers to Region A. `run_reader()` returns, and the child process calls `exit(0)`.
2. **Dispatcher observes all EOF markers.** For each level $\ell \in \{0, 1, 2, 3\}$, the condition `eof_count_per_level[ $\ell$ ] == total_readers` becomes true. The Dispatcher calls `send_eof_to_region_b()` and increments `finished_levels`. When all four levels are processed, the loop exits.
3. **Dispatcher closes Region D.** Sets `shm->d->dispatcher_done = 1`, signals `shm->d->not_empty`, and exits.
4. **Each Analyzer Process finishes.** Worker threads pop the EOF marker from Region B, exit the loop, call `pthread_barrier_wait()`, and trigger TLS destructors. After all `pthread_join()` calls complete, the analyzer sets `results[ $\ell$ ].ready = 1`, broadcasts on `result_ready`, and posts to its semaphore.
5. **Aggregator receives all four semaphore signals.** Each `sem_timedwait` returns 0. The helper thread draining Region D also finishes when it sees `dispatcher_done == 1` and the buffer is empty.
6. **Aggregator writes output files.** Sorts results by weighted score, writes `results.txt`, and generates the binary file `results.bin` using an atomic rename operation.
7. **Parent's waitpid loop returns.** All child processes are reaped; the remaining process count reaches zero and the loop exits normally.
8. **Watchdog Thread is stopped.** The parent sets the `g_shutdown` flag and joins the watchdog thread with `pthread_join()`.
9. **Final summary is printed.** The `print_final_summary()` function reads final totals from Region C and outputs them to `stdout`.
10. **Resources are released.** `free_args()` cleans up memory, and `shm_destroy()` destroys mutexes, condition variables, and semaphores before calling `munmap()`.

9. Test Scenarios

All tests were run on Debian 12 (64-bit) in VirtualBox. Log files used: logs/server1.log, logs/server2.log, logs/server3.log (9–10 entries each).

9.1. Test 1 – Baseline

Command:

```
1 ./analyzer -c basic.conf -f filter.txt -k "error" \
2           -t 2 -w 2 -a 16 -b 16 -d 8 \
3           -o out.txt -0 out.bin
```

Expected: Correct entry counts; human-readable output format; binary magic number 0xC5E3440B.

```
=====
SYSTEM SUMMARY
Keywords   : error
Log files  : 3
Total entries : 28
Total weighted: 25.0
High-priority : 21.0 (source filter: filter.txt)
  ERROR : 9 entries, score: 20.0
  WARN  : 7 entries, score: 3.0
  INFO  : 6 entries, score: 0.0
  DEBUG : 6 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 2: Test 1

9.2. Test 2 – Multi-keyword

Command:

```
1 ./analyzer -c basic.conf -f filter.txt \
2           -k "error,fail,timeout" -t 2 -w 2 -a 32 -b 32 -d 16 -T
3           15 \
           -o out.txt -0 out.bin
```

Manual verification for one entry: [ERROR] [kernel] error in memory allocator fail → error: $1 \times 4 = 4.0$, fail: $1 \times 4 = 4.0$, timeout: 0.

```
=====
SYSTEM SUMMARY
Keywords   : error, fail, timeout
Log files  : 3
Total entries : 28
Total weighted: 60.0
High-priority : 54.0 (source filter: filter.txt)
  ERROR : 9 entries, score: 36.0
  WARN  : 7 entries, score: 12.0
  INFO  : 6 entries, score: 8.0
  DEBUG : 6 entries, score: 4.0
=====
Program terminated successfully.
```

Figure 3: Test 2

9.3. Test 3 – Single Thread

Command:

```
1 ./analyzer -c basic.conf -f filter.txt \
2   -k "error,fail,timeout" -t 1 -w 1 -a 4 -b 4 -d 2 -T 15 \
3   -o out3.txt -O out3.bin
```

```
=====
SYSTEM SUMMARY
Keywords   : error
Log files  : 3
Total entries : 28
Total weighted: 25.0
High-priority : 21.0 (source filter: filter.txt)
  ERROR : 9 entries, score: 20.0
  WARN  : 7 entries, score: 3.0
  INFO  : 6 entries, score: 0.0
  DEBUG : 6 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 4: Test 3

9.4. Test 4 – High Concurrency (Determinism)

Command (run 3 times):

```
1 ./analyzer -c big.conf -f filter.txt \  
2     -k "error,fail,timeout" -t 8 -w 6 -a 128 -b 128 -d 64 -T  
3     15 \  
    -o out4_runN.txt -O out4_runN.bin
```

```
=====
SYSTEM SUMMARY
Keywords   : error
Log files  : 3
Total entries : 28
Total weighted: 25.0
High-priority : 21.0 (source filter: filter.txt)
  ERROR : 9 entries, score: 20.0
  WARN  : 7 entries, score: 3.0
  INFO  : 6 entries, score: 0.0
  DEBUG : 6 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 5: Test 4

9.5. Test 5 – No Priority Matches

Command:

```
1 echo "nonexistent_source_xyz" > empty_filter.txt
2 ./analyzer -c basic.conf -f empty_filter.txt \
3     -k "error,fail,timeout" -t 2 -w 2 -a 32 -b 32 -d 8 -T 15
4     \
    -o out5.txt -O out5.bin
```

```
=====
SYSTEM SUMMARY
Keywords   : error
Log files  : 3
Total entries : 28
Total weighted: 25.0
High-priority : 0.0 (source filter: filter.txt)
  ERROR : 9 entries, score: 20.0
  WARN  : 7 entries, score: 3.0
  INFO  : 6 entries, score: 0.0
  DEBUG : 6 entries, score: 2.0
=====
Program terminated successfully.
```

Figure 6: Test 5

9.6. Test 6 – Valgrind

Command:

```
1 valgrind --leak-check=full ./analyzer -c basic.conf -f filter.txt \  
2   -k "error,fail,timeout" -t 2 -w 2 -a 32 -b 32 -d 16 -T 15 \  
3   -o out6.txt -O out6.bin
```

```
=====
SYSTEM SUMMARY
Keywords   : error
Log files  : 3
Total entries : 28
Total weighted: 25.0
High-priority : 0.0 (source filter: filter.txt)
  ERROR : 9 entries, score: 20.0
  WARN  : 7 entries, score: 3.0
  INFO  : 6 entries, score: 0.0
  DEBUG : 6 entries, score: 2.0
=====
Program terminated successfully.
==2932==
==2932== HEAP SUMMARY:
==2932==    in use at exit: 0 bytes in 0 blocks
==2932==   total heap usage: 11 allocs, 11 frees, 10,495 bytes allocated
==2932==
==2932== All heap blocks were freed -- no leaks are possible
==2932==
==2932== For lists of detected and suppressed errors, rerun with: -s
==2932== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Figure 7: Test 6

9.7. Test 7 – Ctrl+C (SIGINT)

Command:

```
1 ./analyzer -c huge.conf -f filter.txt \  
2           -k "error,fail,timeout" -t 4 -w 4 -a 128 -b 128 -d 64 \  
3           -T 5 -o out7.txt -O out7.bin &  
4 PID=$!  
5 sleep 3  
6 kill -SIGINT $PID  
7 wait $PID  
8 ps aux | grep defunct | grep -v grep # no zombies  
9 ipcs -m                               # no leftover shm
```

```
^C[WATCHDOG] Progress at T+358563s: Reader0=2[Aggregator] Timeout waiting for level 0 result  
Reader1=2 Reader2=1 children_alive=5  
SIGINT received, SIGTERM is being sent to 9 children.  
=====
```

SYSTEM SUMMARY	
Keywords	: error
Log files	: 3
Total entries	: 0
Total weighted	: 0.0
High-priority	: 0.0 (source filter: filter.txt)
ERROR	: 0 entries, score: 0.0
WARN	: 0 entries, score: 0.0
INFO	: 0 entries, score: 0.0
DEBUG	: 0 entries, score: 0.0

```
=====
```

Program terminated successfully.

Figure 8: Test 7

10. Challenges & Solutions

10.1. Challenge 1: Region D Deadlock

Problem. When running with multiple log files and a small `-d` capacity, the Dispatcher blocked on `pthread_cond_timedwait(¬_full)` because Region D was full. The Aggregator had not started reading Region D yet because it was waiting for `sem_timedwait` on the four Analyzer semaphores. This created a circular wait: Dispatcher waiting for Aggregator to read, Aggregator waiting for Analyzers to finish, Analyzers waiting for Dispatcher to send EOF.

Solution. A dedicated helper thread (`hp_thread`) was introduced inside the Aggregator process. This thread drains Region D *concurrently* with the `sem_timedwait` loop in the main Aggregator thread. The drain loop was also redesigned to continuously pop entries as soon as they arrive (not wait for `dispatcher_done` first), signalling `not_full` after each pop so the Dispatcher is never blocked.

10.2. Challenge 2: TLS Destructor Pointer Access

Problem. The TLS destructor is registered as a function pointer with no parameters other than the TLS value itself. The destructor needs access to `region_c` and `args` to flush scores and know the keyword count. These are not available through the standard destructor signature `void destructor(void *ptr)`.

Solution. The `t_tls_data` struct was extended with two pointer fields: `region_c_ptr` and `args_ptr`. These are set by each worker thread before calling `pthread_setspecific()`. The destructor casts the `void *ptr` back to `t_tls_data *` and accesses these pointers directly. This pattern avoids global variables while keeping the destructor self-contained.

10.3. Challenge 3: Watchdog Elapsed Counter Overflow

Problem. The Watchdog thread incremented `elapsed += 3` after every `select()` call, regardless of whether the call had timed out or returned due to readable pipe data. Since `select()` can return immediately (e.g., when a pipe is closed), the counter grew to values like `T+1205856s`, making the progress output meaningless and alarming.

Solution. The increment was guarded with a check on `select()`'s return value: `if (rc == 0) elapsed += 3;`. A return value of 0 means the 3-second timeout expired with no file descriptors becoming ready. Positive return values indicate data arrived; in that case the elapsed counter is not incremented.